

Universidad Nacional de Loja
Facultad Agropecuaria y de Recursos Naturales Renovables
Carrera de Ingeniería Ambiental

GUÍA DE PRÁCTICA N.º 01

Introducción a la programación y fundamentos de Python

Programación y Base de Datos – Unidad 1

Entorno de trabajo: Google Colab

¿Qué aprenderá en esta práctica?

- Usar Google Colab para escribir, ejecutar y guardar programas en Python.
- Mostrar información con `print()` y documentar con comentarios.
- Reconocer y corregir errores de sintaxis, de ejecución y lógicos.
- Trabajar con variables, constantes y los tipos `int`, `float`, `str` y `bool`.
- Aplicar operadores aritméticos, relacionales y lógicos a problemas ambientales.
- Leer datos con `input()`, convertir tipos y presentar resultados con *f-strings*.
- Construir un programa completo *Entrada → Proceso → Salida*.

Docente: Guillermo Chunchu Morocho
Ciclo académico: Septiembre 2026 – Febrero 2027
Cuaderno de trabajo: 01_introduccion_programacion_fundamentos_python.ipynb
Duración estimada: 4 horas (2 en clase + 2 de trabajo autónomo)

Requisitos: cuenta de Google (Gmail o institucional), navegador actualizado y conexión a internet

Índice

1	Datos informativos	2
2	Objetivos	2
2.1	Objetivo general	2
2.2	Objetivos específicos	2
2.3	Resultado de aprendizaje esperado	3
3	Materiales y requisitos previos	3
4	Fundamento teórico breve	3
4.1	¿Qué es programar?	3
4.2	¿Por qué Python?	3
4.3	El modelo Entrada – Proceso – Salida	3
4.4	¿Qué es un cuaderno (<i>notebook</i>) de Colab?	4
5	Preparación del entorno en Google Colab	4
5.1	Paso 1. Ingresar a Google Colab	4
5.2	Paso 2. Abrir el cuaderno de la práctica	4
5.3	Paso 3. Guardar su propia copia	4
5.4	Paso 4. Conocer la interfaz	5
5.5	Paso 5. Ejecutar celdas y atajos de teclado	5
5.6	Paso 6. Prueba de funcionamiento	5
5.7	Uso de la asistencia de IA integrada en Colab	6
6	Desarrollo de la práctica	6
6.1	Actividad 1. Mi primer programa: <code>print()</code> y comentarios	6
6.2	Actividad 2. Los tres tipos de errores	7
6.2.1	2.1 Error lógico: el promedio mal calculado (celda del cuaderno)	8
6.2.2	2.2 Provoque y corrija errores (en celdas nuevas)	8
6.3	Actividad 3. Variables, identificadores y constantes	9
6.4	Actividad 4. Los cuatro tipos de datos primitivos	10
6.5	Actividad 5. Operadores aritméticos y fórmulas ambientales	11
6.6	Actividad 6. Operadores relacionales y lógicos: la regla 30-30-30	12
6.7	Actividad 7. Entrada de datos con <code>input()</code> , conversión de tipos y <i>f-strings</i>	13
6.7.1	7.1 Celda del cuaderno (entrada simulada)	14
6.7.2	7.2 Entrada real desde el teclado en Colab (celda nueva)	14
6.7.3	7.3 Formato con <i>f-strings</i>	15
6.8	Actividad 8. Programa integrado: balance hídrico de una microcuenca	15
7	Buenas prácticas y uso ético de la inteligencia artificial	16
7.1	Buenas prácticas de programación	16
7.2	Uso responsable de la IA	17
8	Micro-retro práctico	17
9	Errores frecuentes y cómo solucionarlos	18
10	Preguntas de autoevaluación	18
11	Entregable y criterios de evaluación	19
11.1	¿Qué entregar?	19
11.2	Rúbrica de evaluación	19

12 Referencias

19

1. Datos informativos

Asignatura	Programación y Base de Datos (2.º ciclo)
Unidad	Unidad 1 – Fundamentos de programación
Tema de la práctica	Introducción a la programación y fundamentos de Python
Modalidad	Práctica guiada en laboratorio / trabajo autónomo
Herramienta	Google Colab (https://colab.research.google.com)
Material base	Cuaderno <code>01_introduccion_programacion_fundamentos_python.ipynb</code>
Contexto de aplicación	Datos ambientales del cantón Loja: precipitación, temperatura, viento, cobertura vegetal y balance hídrico
Nivel de partida	Ninguno: la guía asume que el estudiante nunca ha programado

2. Objetivos

2.1 Objetivo general

Desarrollar programas básicos en Python, ejecutados en Google Colab, que reciban datos ambientales, los procesen mediante expresiones matemáticas y lógicas, y presenten resultados en formato de informe técnico.

2.2 Objetivos específicos

1. Configurar y operar un cuaderno de Google Colab: crear celdas, ejecutarlas, guardarlas y compartirlas.
2. Distinguir los tres tipos de errores de programación y aplicar una estrategia para corregirlos.
3. Declarar variables y constantes con nombres descriptivos según la convención `snake_case`.
4. Identificar el tipo de dato adecuado para cada medición ambiental.
5. Escribir fórmulas ambientales usando correctamente la jerarquía de operadores.
6. Construir condiciones de alerta con operadores relacionales y lógicos.
7. Integrar todo lo aprendido en un programa con estructura Entrada–Proceso–Salida.

2.3 Resultado de aprendizaje esperado

Al terminar la práctica, el estudiante será capaz de **explicar línea por línea** un programa en Python que resuelve un problema ambiental sencillo y de **modificarlo** para un caso nuevo sin ayuda externa.

3. Materiales y requisitos previos

- Computador (o tablet con teclado) con un navegador actualizado: Google Chrome, Microsoft Edge o Mozilla Firefox.
- Cuenta de Google activa (correo Gmail o cuenta institucional habilitada para Google).
- Conexión estable a internet: Colab se ejecuta en servidores de Google, **no en su computador**.
- Archivo del cuaderno de trabajo: `01_introduccion_programacion_fundamentos_python.ipynb`, entregado por el docente.
- Cuaderno de apuntes para registrar dudas, errores encontrados y su solución.

Consejo

No necesita instalar Python para esta práctica. Google Colab ya trae Python y las librerías más usadas en ciencia de datos. La instalación local (VS Code + Python) descrita en la *Guía de instalación del entorno* se usará más adelante en el proyecto integrador.

4. Fundamento teórico breve

4.1 ¿Qué es programar?

Una computadora no “entiende” problemas: solo ejecuta **instrucciones precisas, una tras otra, en el orden en que se escriben**. Programar es traducir la solución de un problema a esas instrucciones.

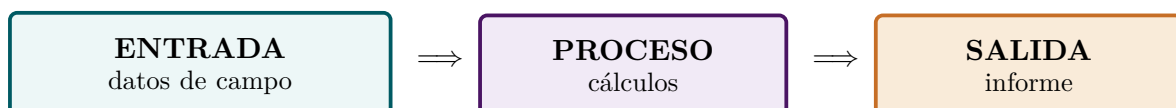
- **Problema:** situación que se desea resolver (p.ej., saber si una microcuenca tiene déficit de agua).
- **Algoritmo:** secuencia finita y ordenada de pasos para resolverlo, escrita en lenguaje humano.
- **Programa:** el algoritmo escrito en un lenguaje de programación (Python) que la computadora puede ejecutar.

4.2 ¿Por qué Python?

Python es un lenguaje **interpretado** (se ejecuta instrucción por instrucción, sin un paso de compilación previo), con una sintaxis cercana al inglés y muy utilizado en ciencias ambientales, hidrología, teledetección y análisis de datos.

4.3 El modelo Entrada – Proceso – Salida

Casi todo programa técnico sigue este esquema, que usaremos durante todo el curso:



4.4 ¿Qué es un cuaderno (*notebook*) de Colab?

Un cuaderno es un documento con extensión `.ipynb` formado por **celdas**:

- **Celdas de texto** (Markdown): explicaciones, títulos, fórmulas. No se ejecutan como código.
- **Celdas de código**: instrucciones Python. Al ejecutarlas, el resultado aparece justo debajo.

Todas las celdas de código comparten la misma memoria (la **sesión** o *entorno de ejecución*). Una variable creada en una celda puede usarse en otra celda posterior, **siempre que la primera ya se haya ejecutado**.

5. Preparación del entorno en Google Colab

5.1 Paso 1. Ingresar a Google Colab

1. Abra el navegador y vaya a <https://colab.research.google.com>.
2. Pulse **Acceder** (esquina superior derecha) e inicie sesión con su cuenta de Google.
3. Si aparece la ventana “Abrir cuaderno”, manténgala abierta: la usaremos en el paso siguiente.

5.2 Paso 2. Abrir el cuaderno de la práctica

Use **una** de las siguientes opciones, según cómo el docente haya compartido el material:

- **Opción A – Subir el archivo desde su computador**: en Colab vaya a **Archivo** → **Subir cuaderno**, seleccione la pestaña **Subir**, pulse **Explorar** y elija `01_introduccion_programacion_fundamentos_python.ipynb`.
- **Opción B – Desde Google Drive**: si el docente compartió el archivo en Drive, ábralo con doble clic y elija **Abrir con** → **Google Colaboratory**.
- **Opción C – Desde GitHub**: en **Archivo** → **Abrir cuaderno**, pestaña **GitHub**, pegue la dirección del repositorio indicada por el docente.

5.3 Paso 3. Guardar su propia copia

¡Muy importante!

El cuaderno que abre por las opciones B o C pertenece al docente. Si no guarda su propia copia, **perderá todo su trabajo** al cerrar la pestaña.

Vaya inmediatamente a **Archivo** → **Guardar una copia en Drive**. Colab abrirá una nueva pestaña con el título “Copia de ...”. **Trabaje siempre en esa copia.**

Luego renombre la copia haciendo clic sobre el título (parte superior izquierda) y use el formato:

`P01_Apellido_Nombre.ipynb` (por ejemplo: `P01_Jaramillo_Ana.ipynb`)

La copia se guarda automáticamente en su Google Drive, en la carpeta **Colab Notebooks**.

5.4 Paso 4. Conocer la interfaz

Elemento	Para qué sirve
Barra de menú	Archivo, Editar, Ver, Insertar, Entorno de ejecución, Herramientas.
Botones + Código / + Texto	Insertan una nueva celda de código o de texto debajo de la celda seleccionada.
Botón Conectar (derecha)	Asigna una máquina virtual de Google para ejecutar el código. Al conectarse muestra los indicadores de RAM y Disco.
Botón <i>play</i> (▶) de cada celda	Ejecuta esa celda.
Indicador [n] a la izquierda	Número de orden en que se ejecutó la celda. Una celda sin número aún no se ha ejecutado.
Panel lateral izquierdo	Índice del cuaderno, búsqueda, variables ({x}) y explorador de archivos (carpeta).

5.5 Paso 5. Ejecutar celdas y atajos de teclado

Acción	Atajo
Ejecutar la celda y quedarse en ella	Ctrl + Enter
Ejecutar la celda y pasar a la siguiente	Shift + Enter
Ejecutar la celda e insertar una nueva debajo	Alt + Enter
Insertar celda de código arriba / abajo	Ctrl + M y luego A / B
Eliminar la celda seleccionada	Ctrl + M y luego D
Convertir celda a texto / a código	Ctrl + M y luego M / Y
Comentar o descomentar líneas	Ctrl + /
Ejecutar todas las celdas	Ctrl + F9

Atención

La primera ejecución de la sesión tarda unos segundos porque Colab debe asignarle una máquina. Si la sesión permanece inactiva un tiempo, Colab la desconecta y **borra las variables de memoria** (no el código). Al volver, use **Entorno de ejecución** → **Ejecutar todas** para reconstruir el estado.

5.6 Paso 6. Prueba de funcionamiento

Pulse + **Código** al final del cuaderno, escriba lo siguiente y ejecútelo con **Shift**+**Enter**:

```
1 | print("Hola, Colab")
2 | 2 + 3
```

SALIDA ESPERADA

```
Hola, Colab
5
```

Consejo

En Colab, si la **última línea** de una celda es una expresión (como `2 + 3`), su valor se muestra automáticamente aunque no use `print()`. Esto **solo ocurre con la última línea**; en un programa `.py` ejecutado fuera de Colab no sucede. Por eso, en esta guía usaremos siempre `print()` para mostrar resultados.

Verifique antes de continuar

- ☐ Estoy trabajando en **mi copia** (P01_Apellido_Nombre), no en el original.
- ☐ El botón **Conectar** muestra los indicadores de RAM/Disco.
- ☐ La celda de prueba muestra “Hola, Colab” y “5”.

5.7 Uso de la asistencia de IA integrada en Colab

Colab puede ofrecer funciones de IA (generación de código, autocompletado en gris mientras escribe, explicación de errores). En esta práctica el objetivo es que **usted** desarrolle el razonamiento. Se recomienda:

- Desactivar las sugerencias automáticas de código desde **Herramientas** → **Configuración** (sección de asistencia de IA o del editor), si su cuenta muestra esa opción.
- No aceptar código generado que no pueda explicar. Revise la sección 7 antes de usar cualquier asistente.

6. Desarrollo de la práctica

Forma de trabajo para cada actividad:

1. Lea la celda de texto “¿Qué vamos a realizar?” del cuaderno.
2. Ejecute la celda de código correspondiente y compare su resultado con la *salida esperada* de esta guía.
3. Lea la explicación detallada y responda mentalmente: *¿por qué se obtuvo ese resultado?*
4. Realice los ejercicios del recuadro **Experimente usted mismo** en **nuevas celdas (+ Código)** debajo de la actividad. No borre el código original.

Consejo

Para agregar sus respuestas escritas, inserte celdas de texto (+ **Texto**) con el título “**Mi respuesta – Actividad N**”. Así el docente encontrará fácilmente su trabajo.

6.1 Actividad 1. Mi primer programa: `print()` y comentarios

¿Qué vamos a lograr? escribir la primera instrucción que comunica un mensaje en pantalla y documentar el código con comentarios.

Paso a paso:

1. Ubique la **Parte 1** del cuaderno y ejecute la celda de código.
2. Observe que las líneas que comienzan con `#` no producen ninguna salida.

```

1  # =====
2  # Mi primer programa en Python
3  # Carrera de Ingeniería Ambiental - Universidad Nacional de Loja
4  # =====
5
6  # La siguiente instrucción envía un texto a la pantalla de salida:
7  print("Bienvenido al curso de Programación y Base de Datos - UNL")
8  print("Iniciando análisis ambiental con Python...")

```

SALIDA ESPERADA

```

Bienvenido al curso de Programación y Base de Datos - UNL
Iniciando análisis ambiental con Python...

```

¿Qué ocurrió?

- # inicia un **comentario**: Python lo ignora; sirve para explicar el código a otras personas (o a usted mismo dentro de unas semanas).
- print(...) es una **función predefinida**: recibe un valor entre paréntesis y lo muestra en pantalla.
- Las comillas "..." delimitan una **cadena de texto (str)**. Pueden ser dobles o simples ('...'), pero deben abrir y cerrar con el mismo tipo.
- Cada print() termina con un salto de línea automático.

Experimente usted mismo

1. Imprima tres líneas con: su nombre completo, su paralelo y el nombre de un río o quebrada de su localidad.
2. Ejecute print("Temperatura:", 19.4, "°C"). ¿Qué hace la coma dentro de print()? ¿Qué separador coloca Python entre los valores?
3. Ejecute print() sin argumentos. ¿Qué observa?

6.2 Actividad 2. Los tres tipos de errores

¿Qué vamos a lograr? reconocer los errores de sintaxis, de ejecución y lógicos, leer el mensaje de error de Colab y corregirlo.

Tipo	¿Cuándo ocurre?	¿Cómo se manifiesta?
Sintaxis	Se rompen las reglas de escritura del lenguaje (paréntesis o comillas sin cerrar, palabras mal escritas).	El programa no llega a ejecutarse . Aparece SyntaxError .
Ejecución	El código está bien escrito, pero pide algo imposible mientras corre (dividir para cero, usar una variable que no existe).	El programa se detiene en la línea del problema: ZeroDivisionError , NameError , TypeError , ValueError ...
Lógico	La fórmula o el razonamiento están mal planteados.	No hay mensaje de error : el programa termina, pero el resultado es incorrecto.

2.1 Error lógico: el promedio mal calculado (celda del cuaderno)

```

1 dia1 = 12.0
2 dia2 = 18.0
3 dia3 = 15.0
4
5 promedio_incorrecto = dia1 + dia2 + dia3 / 3      # Error lógico
6 promedio_correcto = (dia1 + dia2 + dia3) / 3      # Forma correcta
7
8 print("Resultado con error lógico (fórmula mal agrupada):", promedio_incorrecto, "
    mm")
9 print("Resultado correcto:", promedio_correcto, "mm")

```

SALIDA ESPERADA

```

Resultado con error lógico (fórmula mal agrupada): 35.0 mm
Resultado correcto: 15.0 mm

```

Python aplica la **jerarquía de operadores**: la división se resuelve antes que la suma. En la primera fórmula calcula $15.0/3 = 5.0$ y luego $12.0 + 18.0 + 5.0 = 35.0$. Los paréntesis obligan a sumar primero (45.0) y luego dividir (15.0) .

¡Muy importante!

Los errores lógicos son los más peligrosos en ingeniería ambiental: el programa no se detiene, pero entrega cifras falsas a un informe técnico. **Siempre verifique un resultado con un cálculo manual o una estimación aproximada.**

2.2 Provoque y corrija errores (en celdas nuevas)

Cree una celda nueva para **cada** fragmento, ejecútelo, lea el mensaje y luego corrija.

```

1 # a) Error de sintaxis: falta cerrar el paréntesis
2 print("Monitoreo de calidad del aire"

1 # b) Error de ejecución: división para cero
2 numero_dias = 0
3 lluvia_total = 45.0
4 print(lluvia_total / numero_dias)

1 # c) Error de ejecución: variable inexistente (nombre mal escrito)
2 temperatura_media = 18.6
3 print(temperatura_maedia)

```

Cómo leer un mensaje de error en Colab:

1. Vaya a la **última línea** del recuadro rojo: indica el **tipo** de error y una descripción (por ejemplo, `ZeroDivisionError: float division by zero`).
2. Busque la flecha `-->` o el número de línea: señala **dónde** ocurrió.
3. Formule una hipótesis, corrija **un solo cambio** y vuelva a ejecutar.

Consejo

Debajo del error, Colab puede mostrar un botón para *explicar el error* con IA. Úselo solo **después** de intentar entenderlo por su cuenta, y anote con sus palabras la causa del error.

Experimente usted mismo

Registre en una celda de texto una tabla con tres filas (a, b, c) y las columnas: *tipo de error*, *mensaje que mostró Colab*, *causa* y *corrección aplicada*.

6.3 Actividad 3. Variables, identificadores y constantes

¿Qué vamos a lograr? guardar datos de una estación meteorológica en variables con nombres claros y distinguir variables de constantes.

```

1  # Identificadores descriptivos en formato snake_case:
2  estacion_meteorologica = "La Argelia - UNL"
3  altitud_msnm = 2160
4  temperatura_actual_celsius = 19.4
5  lluvia_activa = False
6
7  # Constantes (por convención se escriben en MAYÚSCULAS):
8  GRAVEDAD_ESTANDAR = 9.80665    # m/s^2
9  DENSIDAD_AGUA = 1000.0         # kg/m^3
10
11 print("Estación:", estacion_meteorologica)
12 print("Altitud:", altitud_msnm, "metros sobre el nivel del mar")
13 print("Temperatura:", temperatura_actual_celsius, "°C")
14 print("¿Está lloviendo actualmente?:", lluvia_activa)

```

SALIDA ESPERADA

```

Estación: La Argelia - UNL
Altitud: 2160 metros sobre el nivel del mar
Temperatura: 19.4 °C
¿Está lloviendo actualmente?: False

```

Conceptos clave:

- El signo = es **asignación**, no igualdad matemática: “calcule lo que está a la derecha y guárdelo con el nombre de la izquierda”. Por eso `x = x + 1` es válido en Python.
- **Reglas para nombrar variables:** empiezan con letra o `_`; no llevan espacios, guiones medios, tildes ni ñ (aunque Python 3 acepta tildes, evítelas por compatibilidad); no pueden ser palabras reservadas (`print`, `if`, `for`, `True`...).
- Python **distingue mayúsculas y minúsculas**: `Temperatura`, `temperatura` y `TEMPERATURA` son tres variables distintas.
- **Constantes:** Python no impide modificarlas; escribirlas en MAYÚSCULAS es un acuerdo entre programadores para indicar “este valor no debe cambiar”.
- Incluya la **unidad** en el nombre o en un comentario: `altitud_msnm`, `lluvia_mm`, `area_ha`.

Consejo

Abra el panel **Variables** (ícono {x} en la barra lateral izquierda) para ver todas las variables que existen en la sesión, su tipo y su valor actual.

Experimente usted mismo

1. Indique cuáles de estos nombres son válidos y pruébelo en Colab: `2estacion`, `estacion_2`, `altitud msnm`, `caudal-rio`, `_caudal`, `print`.
2. Cree variables para una estación de su elección (nombre, altitud, temperatura y si hay neblina) e imprímalas.
3. Ejecute en una celda `temperatura_actual_celsius = 21.0` y vuelva a imprimirla. ¿Qué le pasó al valor anterior?

6.4 Actividad 4. Los cuatro tipos de datos primitivos

¿Qué vamos a lograr? identificar el tipo de dato de cada medición ambiental con la función `type()`.

Tipo	Representa	Ejemplo ambiental
<code>int</code>	Números enteros	número de focos de calor: 7
<code>float</code>	Números con decimales	velocidad del viento: 24.85
<code>str</code>	Texto entre comillas	parroquia: "Malacatos"
<code>bool</code>	Verdadero o falso	alerta activada: True

```

1 parroquia = "Malacatos"           # str
2 numero_focos_calor = 7             # int
3 velocidad_viento_kmh = 24.85       # float
4 alerta_incendio_activada = True    # bool
5
6 print("parroquia:", parroquia, "--> Tipo:", type(parroquia))
7 print("numero_focos_calor:", numero_focos_calor, "--> Tipo:", type(
  numero_focos_calor))
8 print("velocidad_viento_kmh:", velocidad_viento_kmh, "--> Tipo:", type(
  velocidad_viento_kmh))
9 print("alerta_incendio_activada:", alerta_incendio_activada, "--> Tipo:", type(
  alerta_incendio_activada))

```

SALIDA ESPERADA

```

parroquia: Malacatos --> Tipo: <class 'str'>
numero_focos_calor: 7 --> Tipo: <class 'int'>
velocidad_viento_kmh: 24.85 --> Tipo: <class 'float'>
alerta_incendio_activada: True --> Tipo: <class 'bool'>

```

Atención

- Los decimales se escriben con **punto**: 24.85. Si escribe 24,85, Python crea una *tupla* de dos enteros (24, 85), no un número decimal.
- True y False se escriben con mayúscula inicial. `true` produce un `NameError`.
- "7" (con comillas) es texto, no número: `type("7")` devuelve `<class 'str'>`.

Experimente usted mismo

1. Para cada dato, decida el tipo y compruébelo con `type()`: pH del suelo (6.8), número de especies observadas (23), nombre del bosque (Bosque Protector El Colambo), ¿la muestra está contaminada? (no), coordenada de latitud (−4.0).

2. Ejecute `print(type(24,85))` y explique el mensaje obtenido. Luego ejecute `dato = 24,85` y `print(type(dato))`.

6.5 Actividad 5. Operadores aritméticos y fórmulas ambientales

¿Qué vamos a lograr? convertir temperaturas, distribuir horas de monitoreo y calcular el área de cobertura de un sensor.

Operador	Operación	Ejemplo	Resultado
+	Suma	12 + 5	17
-	Resta	12 - 5	7
*	Multiplicación	12 * 5	60
/	División real (siempre float)	12 / 5	2.4
//	División entera	12 // 5	2
%	Módulo (residuo)	12 % 5	2
**	Potencia	12 ** 2	144

Jerarquía (de mayor a menor prioridad): () → ** → *, /, //, % → +, -. A igual prioridad, se evalúa de izquierda a derecha.

```

1  temp_celsius = 25.0
2
3  # 1. Conversión de temperatura a Fahrenheit: F = C * 9/5 + 32
4  temp_fahrenheit = (temp_celsius * 9 / 5) + 32
5
6  # 2. Operaciones con horas de monitoreo de campo:
7  total_horas_monitoreo = 58
8  duracion_turno = 8
9  turnos_completos = total_horas_monitoreo // duracion_turno # División entera
10 horas_sobrantes = total_horas_monitoreo % duracion_turno    # Módulo (residuo)
11
12 # 3. Potenciación: área de influencia de un sensor circular (A = pi * r^2)
13 PI = 3.141592
14 radio_cobertura_km = 4.5
15 area_cobertura_km2 = PI * (radio_cobertura_km ** 2)
16
17 print("Temperatura en Celsius:", temp_celsius, "°C")
18 print("Temperatura equivalente en Fahrenheit:", temp_fahrenheit, "°F")
19 print("-----")
20 print("Total de horas de monitoreo:", total_horas_monitoreo)
21 print("Turnos completos de 8 horas:", turnos_completos)
22 print("Horas restantes para el siguiente turno:", horas_sobrantes)
23 print("-----")
24 print("Área de monitoreo del sensor:", area_cobertura_km2, "km²")

```

SALIDA ESPERADA

```

Temperatura en Celsius: 25.0 °C
Temperatura equivalente en Fahrenheit: 77.0 °F
-----
Total de horas de monitoreo: 58
Turnos completos de 8 horas: 7
Horas restantes para el siguiente turno: 2
-----

```

Área de monitoreo del sensor: 63.617238 km²

Análisis:

- $58 // 8 = 7$ porque el 8 cabe 7 veces completas en 58; $58 \% 8 = 2$ porque $7 \times 8 = 56$ y sobran 2 horas.
- La potencia se escribe con ******. El símbolo **^** **no** es potencia en Python (es una operación lógica entre bits): $4.5 \wedge 2$ produce un **TypeError**.
- El módulo **%** sirve, por ejemplo, para saber si un número es par ($n \% 2 == 0$).

Consejo

En lugar de escribir $PI = 3.141592$, Python ofrece el valor exacto en el módulo **math**:

```
1 import math
2 print(math.pi * 4.5 ** 2)
```

Experimente usted mismo

1. Escriba la conversión inversa, de Fahrenheit a Celsius: $C = (F - 32) \times \frac{5}{9}$. Pruebe con $F = 77$ (debe obtener 25.0).
2. Una campaña de muestreo dura 100 horas en turnos de 6 horas. ¿Cuántos turnos completos hay y cuántas horas quedan?
3. Calcule $\frac{12 + 18}{2 \times 3}$ en Python. Escríbalo primero **sin** paréntesis y luego con ellos; explique la diferencia.

6.6 Actividad 6. Operadores relacionales y lógicos: la regla 30-30-30

¿Qué vamos a lograr? construir condiciones de alerta de incendio forestal que devuelvan True o False.

Relacionales		Lógicos	
==	igual a	and	verdadero si todas se cumplen
!=	diferente de	or	verdadero si al menos una se cumple
> / >=	mayor / mayor o igual	not	invierte el valor lógico
< / <=	menor / menor o igual		

Regla 30-30-30 (riesgo crítico de incendio): temperatura $> 30^\circ\text{C}$, humedad relativa $< 30\%$ y viento $> 30\text{ km/h}$, **simultáneamente**.

```
1 temperatura = 31.5      # °C
2 humedad_relativa = 26.0  # %
3 velocidad_viento = 34.0  # km/h
4
5 condicion_temp = temperatura > 30.0
6 condicion_humedad = humedad_relativa < 30.0
7 condicion_viento = velocidad_viento > 30.0
8
9 alerta_critica_30_30_30 = condicion_temp and condicion_humedad and condicion_viento
10 alerta_moderada = condicion_temp or condicion_viento
11
```

```

12 print("¿La temperatura supera los 30 °C?:", condicion_temp)
13 print("¿La humedad es menor al 30 %?:", condicion_humedad)
14 print("¿El viento supera los 30 km/h?:", condicion_viento)
15 print("=====")
16 print("¿ESTADO DE ALERTA CRÍTICA (30-30-30)?:", alerta_critica_30_30_30)
17 print("¿ESTADO DE ALERTA PREVENTIVA MODERADA?:", alerta_moderada)

```

SALIDA ESPERADA

```

¿La temperatura supera los 30 °C?: True
¿La humedad es menor al 30 %?: True
¿El viento supera los 30 km/h?: True
=====
¿ESTADO DE ALERTA CRÍTICA (30-30-30)? True
¿ESTADO DE ALERTA PREVENTIVA MODERADA?: True

```

Tabla de verdad (A y B son condiciones):

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

Atención

No confunda = (asignar un valor) con == (preguntar si dos valores son iguales). Escribir `if temperatura = 30` produce un `SyntaxError`.

Experimente usted mismo

1. Cambie `humedad_relativa` a 45.0 y vuelva a ejecutar. ¿Qué alertas cambian y por qué?
2. Prediga **antes de ejecutar** el resultado con `temperatura = 28.0`, `humedad = 20.0` y `viento = 35.0`. Compruebe en Colab.
3. Cree la variable `condiciones_seguras = not alerta_critica_30_30_30` e imprímala.

Estas condiciones son la base de las estructuras de decisión (`if`, `elif`, `else`) que se estudiarán en la siguiente unidad.

6.7 Actividad 7. Entrada de datos con `input()`, conversión de tipos y *f-strings*

¿Qué vamos a lograr? pedir datos al usuario, convertirlos a número y mostrar resultados con un número controlado de decimales.

¡Muy importante!

Regla de oro: `input()` siempre devuelve texto (`str`), aunque el usuario escriba números. Para hacer cálculos debe convertir el dato con `float()` o `int()`.

7.1 Celda del cuaderno (entrada simulada)

```
1 entrada_lluvia_texto = "14.8" # Esto es lo que recibe input() internamente
2
3 lluvia_mm = float(entrada_lluvia_texto) # Conversión de texto a número
4 lluvia_adicional = 5.2
5 lluvia_total = lluvia_mm + lluvia_adicional
6
7 nombre_sector = "Vilcabamba"
8
9 print(f"Sector evaluado: {nombre_sector}")
10 print(f"Precipitación registrada: {lluvia_mm:.2f} mm")
11 print(f"Precipitación adicional estimada: {lluvia_adicional:.2f} mm")
12 print(f"Precipitación acumulada final: {lluvia_total:.2f} mm")
```

SALIDA ESPERADA

```
Sector evaluado: Vilcabamba
Precipitación registrada: 14.80 mm
Precipitación adicional estimada: 5.20 mm
Precipitación acumulada final: 20.00 mm
```

7.2 Entrada real desde el teclado en Colab (celda nueva)

Ahora use `input()` de verdad. Cree una celda nueva y ejecute:

```
1 nombre_sector = input("Nombre del sector: ")
2 lluvia_texto = input("Precipitación registrada (mm): ")
3
4 print("Tipo de dato recibido:", type(lluvia_texto))
5
6 lluvia_mm = float(lluvia_texto)
7 lluvia_total = lluvia_mm + 5.2
8 print(f"{nombre_sector}: precipitación acumulada = {lluvia_total:.2f} mm")
```

Cómo funciona en Colab:

1. Al ejecutar la celda, aparece un **cuadro de texto** debajo de ella con el mensaje “Nombre del sector:”.
2. Escriba el valor (por ejemplo, Vilcabamba) y pulse **Enter**. Luego aparecerá la segunda pregunta.
3. Mientras Colab espera su respuesta, la celda muestra un ícono girando. **No ejecute otras celdas** hasta responder: quedarán en cola.
4. Si la celda queda “colgada”, pulse el botón de detener (cuadrado) junto a la celda o use **Entorno de ejecución → Interrumpir ejecución**.

Atención

Si escribe el número con coma (14,8), `float()` producirá `ValueError: could not convert string to float`. Escriba siempre los decimales con **punto**.

7.3 Formato con *f-strings*

Expresión	Significado	Ejemplo con 102600.456
{x}	valor tal cual	102600.456
{x:.2f}	2 decimales	102600.46
{x:.1f}	1 decimal	102600.5
{x:,.2f}	separador de miles y 2 decimales	102,600.46
{x:.0f}	sin decimales (redondeado)	102600

Experimente usted mismo

1. Ejecute `print("14.8" + "10")` y `print(float("14.8") + float("10"))`. Explique la diferencia (concatenación frente a suma).
2. Modifique la celda 7.2 para que pida también el número de días de lluvia (use `int()`) y muestre la lluvia promedio por día con 1 decimal.
3. Ingrese a propósito la palabra mucho como precipitación. ¿Qué tipo de error aparece? ¿Es de sintaxis, ejecución o lógico?

6.8 Actividad 8. Programa integrado: balance hídrico de una microcuenca

¿Qué vamos a lograr? integrar variables, operadores, condiciones y formato de salida en un programa completo de ingeniería.

Planteamiento del problema:

- **Entradas:** nombre de la microcuenca, precipitación mensual P (mm), evapotranspiración potencial ETP (mm) y área (ha).
- **Proceso:**
 1. Balance hídrico neto: $BH = P - ETP$.
 2. Déficit: existe si $BH < 0$.
 3. Volumen: $V = BH \times \text{Área} \times 10$, porque 1 mm de lámina de agua sobre 1 ha equivale a 10 m^3 ($0.001 \text{ m} \times 10\,000 \text{ m}^2$).
- **Salida:** informe técnico en pantalla.

```

1  # 1. ENTRADA DE DATOS:
2  nombre_cuenca = "Microcuenca Malacatos - Sector El Carmen"
3  precipitacion_mm = 65.4      # Lluvia en el mes (mm)
4  evapotranspiracion_mm = 88.2 # Pérdida de agua por evaporación y plantas (mm)
5  area_hectareas = 450.0      # Área de captación (ha)
6
7  # 2. PROCESAMIENTO:
8  balance_hidrico_mm = precipitacion_mm - evapotranspiracion_mm
9  existe_deficit = balance_hidrico_mm < 0.0
10 volumen_netto_m3 = balance_hidrico_mm * area_hectareas * 10.0
11
12 # 3. SALIDA DE RESULTADOS:
13 print("=====")
14 print("          INFORME TÉCNICO DE BALANCE HÍDRICO MENSUAL          ")
15 print("          Carrera de Ingeniería Ambiental - UNL          ")
16 print("=====")
17 print(f"Zona de estudio:          {nombre_cuenca}")

```

```

18 print(f"Área territorial:           {area_hectareas:,.1f} hectáreas")
19 print(f"Precipitación mensual (P): {precipitacion_mm:.1f} mm")
20 print(f"Evapotranspiración (ETP):   {evapotranspiracion_mm:.1f} mm")
21 print("-----")
22 print(f"Balance Hídrico Neto:         {balance_hidrico_mm:.2f} mm")
23 print(f"¿Existe déficit de agua?:     {existe_deficit}")
24 print(f"Volumen hídrico resultante:   {volumen_netom3:,.2f} m³ de agua")
25 print("=====")

```

SALIDA ESPERADA

```

=====
                INFORME TÉCNICO DE BALANCE HÍDRICO MENSUAL
                Carrera de Ingeniería Ambiental - UNL
=====
Zona de estudio:           Microcuenca Malacatos - Sector El Carmen
Área territorial:           450.0 hectáreas
Precipitación mensual (P): 65.4 mm
Evapotranspiración (ETP):  88.2 mm
-----
Balance Hídrico Neto:      -22.80 mm
¿Existe déficit de agua?:  True
Volumen hídrico resultante: -102,600.00 m³ de agua
=====

```

Verificación manual del resultado (hábito obligatorio del ingeniero):

$$BH = 65.4 - 88.2 = -22.8 \text{ mm} \quad V = -22.8 \times 450 \times 10 = -102\,600 \text{ m}^3$$

El signo negativo indica que en ese mes se pierde más agua por evapotranspiración de la que ingresa por lluvia: **déficit hídrico**.

Experimente usted mismo

1. Copie el programa en una celda nueva y reemplace las cuatro entradas por `input()` (con las conversiones necesarias).
2. Ejecútelo para un mes lluvioso: $P = 180.0 \text{ mm}$, $ETP = 95.5 \text{ mm}$, área = 320 ha. Verifique a mano el balance y el volumen.
3. Agregue al informe una línea con el volumen expresado en litros ($1 \text{ m}^3 = 1000 \text{ L}$).

Verifique antes de continuar

- ☐ Todas las celdas del cuaderno tienen número [n] (se ejecutaron sin errores).
- ☐ Cada actividad tiene debajo sus celdas de *Experimente usted mismo* resueltas.
- ☐ Mis respuestas escritas están en celdas de texto tituladas “Mi respuesta – Actividad N”.

7. Buenas prácticas y uso ético de la inteligencia artificial

7.1 Buenas prácticas de programación

- Use nombres de variables descriptivos y con unidades: `caudal_m3s` mejor que `c`.
- Comente el **porqué** de una fórmula, no lo obvio.
- Organice cada programa en bloques: *entrada*, *proceso* y *salida*.
- Verifique los resultados con un cálculo manual o una estimación de orden de magnitud.

- Antes de entregar, use **Entorno de ejecución → Reiniciar sesión y ejecutar todo** para confirmar que el cuaderno funciona de principio a fin, en orden.

7.2 Uso responsable de la IA

Las herramientas de IA (Gemini en Colab, ChatGPT, Claude, Copilot) son asistentes valiosos si se usan con criterio técnico.

Uso adecuado	Uso inadecuado
Pedir que explique un mensaje de error que no comprende: “¿Por qué ocurre este <code>TypeError</code> en mi línea 4?”.	Copiar y pegar código que no puede explicar.
Solicitar analogías: “Explícame la diferencia entre <code>int</code> y <code>float</code> con un ejemplo ambiental”.	Pedir que resuelva los micro-retos, trabajos o el proyecto sin su participación.
Comparar su solución, ya terminada, con una alternativa y analizar cuál es más legible.	Aceptar el autocompletado de Colab sin leerlo.

¡Muy importante!

En la defensa individual se le pedirá explicar cualquier línea de su código. Si no puede explicar lo que entregó, la calificación de ese componente será **cero**.

8. Micro-reto práctico

Consigna. En una celda nueva al final del cuaderno, escriba un programa que calcule el **índice de densidad de vegetación** de una parcela de reforestación de **12 hectáreas** en la que se contabilizaron **3 840 árboles** nativos de *Alnus acuminata* (aliso).

1. Defina las variables `nombre_especie`, `area_ha` y `total_arboles` con el tipo de dato adecuado.
2. Calcule la densidad:

$$\text{densidad} = \frac{\text{total_arboles}}{\text{area_ha}} \quad [\text{árboles/ha}]$$
3. Evalúe si la densidad es óptima: se considera óptima si **supera** los 250 árboles/ha (resultado `True/False`).
4. Imprima un reporte formateado con *f-strings* que muestre la especie, la densidad con 1 decimal y si cumple la condición.

Extensión (opcional): modifique el programa para que los tres datos se ingresen con `input()` y calcule cuántos árboles faltarían plantar para alcanzar 400 árboles/ha.

Consejo

Antes de escribir código, escriba el algoritmo en una celda de texto con sus palabras (Entrada, Proceso, Salida) y estime el resultado a mano. Luego compare con lo que obtiene Python.

9. Errores frecuentes y cómo solucionarlos

Mensaje o síntoma	Causa probable y solución
SyntaxError: '(' was never closed	Falta cerrar un paréntesis. Cuente los paréntesis de apertura y cierre.
SyntaxError: unterminated string literal	Comillas sin cerrar o mezcladas ("texto").
NameError: name '...' is not defined	Variable mal escrita, o definida en una celda que aún no se ha ejecutado. Ejecute las celdas anteriores en orden.
TypeError: can only concatenate str (not "float") to str	Se intentó sumar texto con número. Convierta con <code>float()</code> o use <i>f-strings</i> .
ValueError: could not convert string to float	Se ingresó texto o un decimal con coma en <code>input()</code> . Use punto decimal.
ZeroDivisionError	División entre cero. Revise el valor del divisor.
IndentationError: unexpected indent	Espacios al inicio de la línea. En esta unidad, todas las líneas empiezan en la columna 1.
Resultado numérico absurdo, sin error	Error lógico: revise paréntesis y jerarquía de operadores, unidades y fórmula.
La celda no termina (ícono girando)	Está esperando un <code>input()</code> o hay otra celda en ejecución. Responda o interrumpa la ejecución.
“Sesión desconectada” / variables perdidas	Colab cerró la sesión por inactividad. Vuelva a conectar y use Ejecutar todas .

10. Preguntas de autoevaluación

Responda en una celda de texto al final del cuaderno:

1. ¿Cuál es la diferencia entre un algoritmo y un programa?
2. ¿Por qué un error lógico es más peligroso que un error de sintaxis en un informe ambiental?
3. ¿Qué valor devuelve `10 / 4`? ¿Y `10 // 4`? ¿Y `10 % 4`? Justifique.
4. ¿Qué tipo de dato usaría para: número de muestras de agua, concentración de nitratos (mg/L), nombre del río, presencia de coliformes?
5. Explique por qué `input()` requiere conversión antes de realizar cálculos.
6. ¿Qué diferencia hay entre `=` y `==`?
7. Con temperatura = 33 °C, humedad = 35 % y viento = 40 km/h, ¿se activa la alerta crítica 30-30-30? ¿Y la moderada? Justifique sin ejecutar código.
8. ¿Qué significa `{volumen:,.2f}` dentro de una *f-string*?
9. ¿Qué sucede con las variables de un cuaderno de Colab cuando la sesión se desconecta?

11. Entregable y criterios de evaluación

11.1 ¿Qué entregar?

1. El cuaderno `P01_Apellido_Nombre.ipynb` con:
 - todas las celdas originales ejecutadas;
 - los ejercicios *Experimente usted mismo* de las actividades 1 a 8;
 - la tabla de errores de la actividad 2;
 - el micro-reto resuelto;
 - las respuestas de autoevaluación.
2. **Forma de entrega:** descargue el archivo con **Archivo** → **Descargar** → **Descargar .ipynb** y súbalo a la plataforma indicada por el docente, o comparta el enlace con **Compartir** (botón superior derecho), asignando permiso de *Lector* al correo del docente.

Atención

Antes de entregar ejecute **Entorno de ejecución** → **Reiniciar sesión y ejecutar todo**. Si alguna celda falla, corríjala. Un cuaderno que solo funciona ejecutando las celdas en un orden “especial” no es un programa correcto.

11.2 Rúbrica de evaluación

Criterio	Excelente	Satisfactorio	En desarrollo
Funcionamiento del cuaderno	Todo se ejecuta en orden, sin errores, tras reiniciar la sesión.	Errores menores en uno o dos ejercicios.	Varias celdas fallan o no se ejecutaron.
Ejercicios <i>Experimente</i>	Todos resueltos y con explicación de lo observado.	La mayoría resueltos; explicaciones breves.	Incompletos o sin explicación.
Micro-reto	Correcto, con nombres claros, comentarios y formato con <i>f-strings</i> .	Correcto, pero con formato o nombres mejorables.	Resultado incorrecto o incompleto.
Comprensión de errores	Tabla completa; identifica tipo, causa y corrección.	Tabla con alguna imprecisión.	No identifica correctamente los tipos de error.
Autoevaluación y defensa	Respuestas precisas; explica cualquier línea de su código.	Respuestas correctas con vacíos menores.	No logra explicar su propio código.

12. Referencias

- Python Software Foundation. (2025). *El tutorial de Python*. <https://docs.python.org/es/3/tutorial/>
- Google. (s.f.). *Colaboratory: preguntas frecuentes*. <https://research.google.com/colaboratory/faq.html>
- Downey, A. B. (2024). *Think Python: How to think like a computer scientist* (3.^a ed.). O'Reilly Media. <https://allendowney.github.io/ThinkPython/>

- Matthes, E. (2023). *Python Crash Course* (3.^a ed.). No Starch Press.